Implementing Task Line Formatting Function

https://gemini.google.com/app/ba5c0929c82816c2

User prompt: task_manager.py: ```python from datetime import date def add_task(tasks, title, due_date=None, priority="medium"): """Add a new task to the task list. Args: tasks (list): The list of existing task dictionaries. title (str): The task's title. due_date (date, optional): The task's due date. Defaults to None. priority (str): One of "high", "medium", "low". Defaults to "medium". Returns: list: The updated list of tasks. """ tasks.append({"title": title, "due_date": due_date, "priority": priority, "completed": False}) return tasks def mark_complete(tasks, title): """Mark a task as completed by title. Args: tasks (list): The list of task dictionaries. title (str): The title of the task to mark complete. Returns: bool: True if a matching task was found and updated, False otherwise. """ for task in tasks: if task["title"] == title: task["completed"] = True return True return False def get_overdue_tasks(tasks, current_date): overdue = [] for task in tasks: if task["due_date"] < current_date and not task["completed"]: overdue.append(task) return overdue def calculate_completion_rate(tasks): if not tasks: return 0.0 completed = sum(1 for t in tasks if t["completed"]) return completed / len(tasks) ``` The existing utils.py provides the following implementation (docstring abbreviated here; logic unchanged): ```python def due_label(due, today): """Return no due date, due today, due in N day(s), or overdue by N day(s)."""" if due is None: return "no due date" delta = (due - today).days if delta == 0: return "due today" if delta > 0: return f"due in {delta} day" + ("" if delta == 1 else "s") return f"overdue by {-delta} day" + ("" if delta == -1 else "s") ``` Implement format_task_line(task, today). Import and call utils.due_label(task["due_date"], today) for every task, including None dates; do not invent another helper or duplicate its logic. Return one line in the format TITLE [PRIORITY] - DUE_LABEL, preserving the title and lowercase priority exactly and the helper result verbatim. Assume valid task dictionaries with single-line titles. Do not modify the task or utils.py. Include a Google-style docstring with Args: and Returns:. Return only the necessary import and function in one Python code block.

Response:

```
import utils

def format_task_line(task, today):
    """Format a task dictionary into a single descriptive string line.

    Args:
        task (dict): A task dictionary containing 'title', 'priority', and 'due_date'.
        today (date): The current date used to compute the due status label.

    Returns:
        str: A formatted string in the format TITLE [PRIORITY] - DUE_LABEL.
    """
    label = utils.due_label(task["due_date"], today)
    return f"{task['title']} [{task['priority']}] - {label}"
```
